ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

DevOps & Software Engineering — 2026/2027

TD 3 — Tutoriel – DevOps Lab

Jira, GitHub, VS Code, CI/CD et Node.js

Ce lab simule un véritable workflow DevOps pour une petite équipe de développement. Il vous guide à travers tout le cycle de vie du développement logiciel : de la planification du projet dans Jira, au développement collaboratif avec GitHub + VS Code, aux builds automatisés avec GitHub Actions, et à l'assurance qualité grâce aux tests unitaires + d'intégration avec Vitest et Supertest.

Vous travaillerez en équipe de trois à quatre développeurs, chacun responsable d'une fonctionnalité différente d'un projet Node.js partagé.

Le code source de base est fourni dans l'archive ZIP :

Full_DevOps_Lab.zip

Objectifs d'apprentissage

À la fin de ce lab, vous serez capable de :

- **PLAN :** structurer un projet Scrum dans Jira Cloud, créer des Epics et des User Stories, et les relier à de vrais commits et pull requests grâce à l'intégration GitHub for Jira.
- **CODE :** développer de manière collaborative dans VS Code en utilisant des branches Git, des messages de commit clairs et des pull requests associées aux tickets Jira.
- **BUILD :** configurer et analyser un pipeline CI/CD GitHub Actions qui automatise le lint, les tests et la vérification de la couverture pour chaque commit et chaque PR.
- **TEST :** exécuter et interpréter les suites Vitest + Supertest, maintenir des seuils minimaux de couverture (≥ 80 %), et vérifier que chaque module fonctionne correctement avant la fusion dans la branche principale.

Présentation du projet

Chaque étudiant de l'équipe de 4 personnes est responsable d'une User Story Jira et du module de code correspondant dans le dépôt partagé.

Étudiant	Story Jira	Fichier du projet	Fonctionnalité
A	DEVOPS-1	src/routes/auto/version.route.js	Retourner la version de l'application
B	DEVOPS-2	src/routes/auto/info.route.js	Retourner les informations d'exécution
C	DEVOPS-3	src/routes/auto/boom.route.js	Simuler une erreur de l'API
D	DEVOPS-4	src/utils/status.js	Retourner le libellé d'état selon le temps de fonctionnement

Les quatre stories forment ensemble le cœur de l'API de l'application, qui sera testée et validée par la CI.

Résumé du déroulement du lab

Étape	Phase	Outils principaux	Livrables
1 – PLAN	Organisation du projet	Jira Software Cloud	Epics, User Stories, lien Jira–GitHub
2 – CODE	Développement collaboratif	GitHub, VS Code	Branches de fonctionnalité, commits, PR
3 – BUILD	Intégration continue	GitHub Actions	Pipeline automatisé lint + tests
4 – TEST	Assurance qualité	Vitest, Supertest	Couverture de 80 %, vérifications CI vertes

Étape 1 — PLAN (Jira Cloud)

Structurez le projet DevOps dans Jira pour gérer la collaboration, suivre l'avancement et relier chaque fonctionnalité directement aux commits, branches et pull requests dans GitHub.

Chaque étudiant prendra en charge une User Story liée à un vrai fichier du projet Node.js partagé (voir le ZIP dans le DVL).

Instructions détaillées

1. Créer un projet Jira Software (Scrum)

- Allez sur : https://www.atlassian.com/software/jira
- Choisissez : **Software → Scrum → Create Project**
- Nom du projet : **DevOps Lab Node.js Project**
- Clé du projet : **DEVOPS** (elle servira de préfixe aux identifiants des tickets, par exemple DEVOPS-3)

2. Configurer le tableau de l'équipe

- Allez dans **Board Settings → Columns**
- Créez le workflow :
- To Do → In Progress → Review → Done
- Ajoutez les 4 étudiants au même tableau du projet.

Chaque ticket Jira (User Story) passera d'une colonne à l'autre au fur et à mesure de l'avancement des commits et des PR.

3. Créer les Epics

Epic	Description
API Core Development	Endpoints pour /version, /info, /boom, /health
Error Handling	Gestion des exceptions et middleware global

Monitoring	Temps de fonctionnement de l'application, informations d'exécution et libellés d'état
CI/CD Automation	Lint, tests, couverture et pipeline GitHub Actions

Chaque User Story (ci-dessous) doit être rattachée à l'Epic **API Core Development**.

4. Ajouter les User Stories (4 minimum)

Chaque User Story correspond à la fonctionnalité attribuée à un étudiant dans le projet ZIP.

ID de la story	Description	Critères d'acceptation
DEVOPS-1	Implémenter l'endpoint /version	- GET /version retourne { "version": "x.x.x" } - HTTP 200 - Le test d'intégration passe
DEVOPS-2	Implémenter l'endpoint /info	- GET /info retourne { name, version, node, uptime } - Réponse JSON - Test unitaire pour les fonctions utilitaires (helpers)
DEVOPS-3	Implémenter l'endpoint /boom	- GET /boom déclenche une erreur 500 - Le gestionnaire d'erreurs global retourne { error: true, message }
DEVOPS-4	Implémenter l'utilitaire formatStatus()	- La fonction retourne warming-up, healthy ou steady selon le temps de fonctionnement (uptime) - Le test unitaire couvre toutes les branches

Chaque story correspond directement à un fichier du **projet ZIP** :

Étudiant	Story Jira	Fichier à modifier	Rôle
A	DEVOPS-1	src/routes/auto/version.route.js	Développer la route /version
B	DEVOPS-2	src/routes/auto/info.route.js	Développer la route /info
C	DEVOPS-3	src/routes/auto/boom.route.js	Ajouter la route de simulation d'erreur
D	DEVOPS-4	src/utils/status.js	Améliorer l'utilitaire de libellé d'état

ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

5. Exemple d'une story complète

DEVOPS-1 — Créer l'endpoint API /version

En tant que développeur,
je veux fournir une route API /version,
afin que l'application puisse exposer son numéro de version actuel.

Critères d'acceptation :
- GET /version retourne un JSON avec la clé "version"
- Code de réponse = 200
- Le test d'intégration test/version.test.js doit passer
- Le commit et la branche contiennent DEVOPS-1

La story passe à « Done » quand la PR est fusionnée et que la CI réussit.

6. Intégrer Jira avec GitHub

1. Dans Jira → **Apps → Find new apps → Search « GitHub for Jira (Atlassian) »**
2. Installez et autorisez votre organisation GitHub.
3. Connectez votre dépôt (**Full_DevOps_Lab_Project_Final_Commented_EN.zip** → une fois déposé sur GitHub).

Une fois l'intégration terminée :

- Une branche nommée **feature/DEVOPS-2-info-endpoint** apparaît automatiquement dans Jira lorsqu'elle est poussée.
- Les commits comme : **feat(DEVOPS-2): add /info endpoint** seront liés automatiquement à la story Jira.
- Les Pull Requests intitulées : **DEVOPS-2 | Implement /info route** apparaîtront dans le panneau **Development** du ticket Jira correspondant.

Étape 2 — CODE (VS Code + GitHub)

Organisez la phase de développement collaboratif : chaque étudiant travaille sur sa fonctionnalité dans le **projet Node.js partagé** (Full_DevOps_Lab_Project_Final_Commented_EN.zip).

L'objectif est d'**implémenter, tester et committer** les modifications en utilisant **Git**, **VS Code** et les **conventions Jira**.

ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

Instructions détaillées

1. Cloner le projet depuis GitHub

Une fois que l'enseignant a déposé le projet ZIP dans l'organisation GitHub de votre classe :

```
git clone https://github.com/<org>/<repo>.git
cd <repo>
npm install --legacy-peer-deps
npm ci --legacy-peer-deps
```

Fichiers attendus à la racine :

```
package.json
src/
test/
.github/workflows/ci.yml
README.md
```

Si vous voyez cette structure, le projet est correctement initialisé.

2. Configurer l'environnement VS Code

Ouvrez le dossier dans **VS Code**, puis installez :

Extension	**Rôle**
ESLint	Met en évidence les erreurs de syntaxe et de style
GitLens	Affiche les informations sur les commits et les branches
GitHub Pull Requests & Issues	Ouvrir des PR directement dans VS Code
Jira and Bitbucket (Atlassian Labs)	Relier VS Code aux tickets Jira

3. Créer votre branche pour la User Story attribuée

Chaque étudiant doit créer une branche à partir de main en respectant la règle de nommage :

```
git checkout -b feature/DEVOPS-<story-id>-<short-desc>
```

Étudiant	**Story Jira**	**Exemple de branche**
A	DEVOPS-1	feature/DEVOPS-1-version-endpoint
B	DEVOPS-2	feature/DEVOPS-2-info-endpoint
C	DEVOPS-3	feature/DEVOPS-3-boom-endpoint
D	DEVOPS-4	feature/DEVOPS-4-status-utility

Cela permet à Jira de relier automatiquement votre branche à votre story.

ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

4. Modifier le bon fichier du projet (correspondance avec le ZIP)

Chaque étudiant travaille sur un fichier précis du projet ZIP. Ouvrez ces chemins dans VS Code :

Étudiant	Chemin du fichier (dans le projet ZIP)	Description de la tâche
A	src/routes/auto/version.route.js	Implémenter l'endpoint /version → retourne la version du package
B	src/routes/auto/info.route.js	Implémenter l'endpoint /info → retourne le nom, la version et le temps de fonctionnement (uptime) de l'application
C	src/routes/auto/boom.route.js	Implémenter l'endpoint /boom → simuler une erreur 500 pour les tests
D	src/utils/status.js	Implémenter formatStatus() → convertir le temps de fonctionnement (en secondes) en libellés

Astuce :

Vous pouvez ouvrir l'explorateur de fichiers de VS Code → src/ → suivre ce chemin pour atteindre votre fichier.

Toutes les fonctions et tous les commentaires sont déjà préparés ; vous devez seulement vérifier ou améliorer la logique.

5. Build et exécution en local

Avant de committer, vérifiez que le serveur fonctionne :

```
npm run dev
```

Sortie attendue :

```
[server] listening on http://localhost:3000
```

Testez votre endpoint dans le navigateur ou avec curl :

```
http://localhost:3000/version
http://localhost:3000/info
http://localhost:3000/boom
```

6. Lancer les tests en local

Chaque endpoint possède déjà des tests d'intégration et des tests unitaires dans le ZIP, sous test/.

Endpoint	Fichier de test d'intégration	Fichier de test unitaire
/version	test/version.test.js	—
/info	test/info.test.js	test/unit/appInfo.test.js

/boom	test/boom.test.js	test/unit/errorHandler.test.js
/health & utilitaire Status	test/health.test.js	test/unit/status.test.js

Lancer tous les tests :

```
npm test
```

Lancer avec la couverture :

```
npm test -- --coverage

Couverture attendue :
Lignes (Lines) : ≥ 80%
Branches : ≥ 70%
```

Si un test échoue, ouvrez le fichier indiqué et corrigez la logique ou le test.

7. Préparer (stage) et committer avec la référence Jira

Une fois que votre fonctionnalité fonctionne et que les tests passent :

```
git add .
git commit -m "feat(DEVOPS-2): add /info endpoint"
git push origin feature/DEVOPS-2-info-endpoint
```

Règles :

- Toujours inclure la clé Jira dans le commit (feat(DEVOPS-#): …)
- Un commit par modification logique (par exemple : code → test → correction)
- Les commits sans clé Jira ne seront pas liés aux tickets dans Jira.

8. Ouvrir une Pull Request (PR)

Dans GitHub :

1. Cliquez sur **Compare & Pull Request**.
2. Titre : DEVOPS-2 | Implement /info endpoint
3. Ajoutez une courte description : ce qui a été fait ; quels tests ont été mis à jour ou créés ; capture d'écran (facultatif).
4. Désignez un autre membre de l'équipe pour la revue.
5. Fusionnez uniquement lorsque le pipeline CI réussit (GitHub Actions).

9. Commandes communes pour tous

Action	**Commande**	**Description**

Démarrer le serveur	npm run dev	Lance l'application sur http://localhost:3000
Lancer les tests	npm test	Exécute les tests unitaires + d'intégration
Lancer la couverture	npm test -- --coverage	Génère le rapport de couverture dans /coverage/
Linter le code	npm run lint	Vérifie le style et la syntaxe du code
Voir le rapport de couverture	open coverage/index.html	Ouvre le rapport HTML en local

10. Livrables attendus à la fin de l'étape 2

Élément	Résultat attendu
Une branche créée par étudiant (feature/DEVOPS-X-...)	✅
Le bon fichier modifié selon l'attribution	✅
Le code s'exécute en local (npm run dev)	✅
Les tests passent et la couverture est ≥ 80 %	✅
Le commit contient la clé Jira et une description	✅
Pull request créée et liée à la story Jira	✅

Étape 3 — BUILD (GitHub Actions CI/CD)

Automatisez les tests, le lint et la validation du code avec **GitHub Actions**.

Chaque push ou Pull Request doit déclencher automatiquement le pipeline CI pour garantir la qualité du code et la couverture avant la fusion.

Cette étape relie les **builds locaux** (npm run lint, npm test) au **pipeline CI** défini dans le projet ZIP sous : .github/workflows/ci.yml

Instructions détaillées

1. Localiser le fichier de configuration de la CI

Dans votre projet ZIP, ouvrez le fichier suivant :

```
.github/
└── workflows/
    └── ci.yml
```

Ce fichier définit le **workflow d'intégration continue** qui s'exécute automatiquement sur GitHub à chaque fois que du code est poussé ou qu'une Pull Request est créée.

2. Comprendre la logique du workflow CI

Le fichier .github/workflows/ci.yml contient déjà un workflow complet :

ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

```yaml
name: CI

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Test (with coverage)
        run: npm test -- --coverage

      - name: Upload coverage artifact
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: coverage-report
          path: coverage
```

💡 **Explication (ligne par ligne) :**

Section	Rôle
on:	Déclenche le workflow quand du code est poussé ou qu'une PR est ouverte sur main
jobs:	Définit un job nommé build-and-test

runs-on:	Utilise Ubuntu (Linux) comme environnement du runner
checkout@v4	Clone votre dépôt GitHub dans l'environnement de CI
setup-node@v4	Installe Node.js (v20) et active le cache pour des builds plus rapides
npm ci	Installe toutes les dépendances définies dans package.json
npm run lint	Vérifie la mise en forme et la cohérence de la syntaxe du code
npm test -- --coverage	Exécute tous les tests (unitaires + intégration) et génère le rapport de couverture
upload-artifact	Enregistre le dossier coverage comme artefact téléchargeable dans GitHub Actions

3. Vérifier la cohérence en local avant de pousser

Assurez-vous toujours que la CI réussira en testant d'abord en local :

```
npm run lint
npm test -- --coverage
```

Résultat attendu :

Tests : réussis

Résumé de la couverture : Lignes ≥ 80 %, Branches ≥ 70 %

Si vous obtenez des erreurs de lint, corrigez-les avant de pousser.

4. Pousser votre branche pour déclencher le workflow

```
git push origin feature/DEVOPS-2-info-endpoint
```

Ensuite, allez sur votre dépôt GitHub → onglet **Actions**.

Vous devriez voir le workflow **CI** démarrer automatiquement :

build-and-test — en cours

Une fois terminé :

- ✅ **Coche verte** → toutes les étapes ont réussi
- ❌ **Croix rouge** → un ou plusieurs tests ont échoué ou la couverture est trop faible

5. Valider les seuils de couverture des tests

Ouvrez le fichier package.json à la racine de votre projet.

Descendez jusqu'à la section "vitest" — elle impose des niveaux minimaux de couverture :

```
"vitest": {
  "coverage": {
    "provider": "v8",
    "all": true,
    "include": ["src/**/*.js"],
    "lines": 80,
    "functions": 80,
    "branches": 70,
    "statements": 80
  }
}
```

Si la couverture est inférieure à ces seuils, la CI échouera automatiquement.

Pour corriger :

- Ajoutez des tests unitaires dans test/unit/*.test.js
- Améliorez la couverture de la logique (par exemple : tester les cas négatifs dans status.js)
- Relancez : npm test -- --coverage en local jusqu'à atteindre ≥ 80 %

6. Télécharger et vérifier l'artefact de couverture (sur GitHub)

Une fois la CI terminée :

1. Allez dans **Actions → CI → Summary**
2. Descendez jusqu'à **Artifacts**
3. Cliquez sur **coverage-report.zip** → Download
4. Décompressez-le et ouvrez index.html pour visualiser la couverture par fichier.

Vous devriez voir :

- Des barres de couverture vertes (≥ 80 %)
- Chaque fichier de src/ listé avec son score de tests

7. Dépannage courant

Problème	**Cause possible**	**Correction**
La CI échoue sur « npm run lint »	ESLint a trouvé des problèmes de style	Lancez npm run lint --fix en local
La CI échoue sur « npm test »	Certains tests ont échoué	Vérifiez la sortie de la console, relancez en local
Couverture < 80 %	Branche ou tests négatifs manquants	Ajoutez des tests unitaires
« Actions not triggered »	Mauvaise branche (ni main ni PR)	Fusionnez la PR dans main ou faites un rebase

« Permission denied »	Dépôt non autorisé dans l'application Jira/GitHub	Vérifiez les permissions de l'intégration

8. Vérifier les résultats de la CI dans Jira

Une fois le workflow terminé avec succès, allez dans Jira → ouvrez votre User Story (par exemple DEVOPS-2) :

Dans le panneau **Development**, vous devriez voir :

- Branche liée (feature/DEVOPS-2-info-endpoint)
- 1 commit (feat(DEVOPS-2): add /info endpoint)
- 1 Pull Request (fusionnée)
- **Statut du build :** ✔ Passed (depuis GitHub Actions)

Cela confirme l'intégration de bout en bout entre **Jira**, **GitHub** et la **CI/CD**.

9. Liste de vérification de fin d'étape

Élément à valider	**Résultat attendu**
.github/workflows/ci.yml présent et fonctionnel	✅
npm run lint en local réussit	✅
npm test -- --coverage en local réussit (≥ 80 %)	✅
Pipeline GitHub Actions déclenché automatiquement	✅
Toutes les étapes de CI vertes dans l'onglet Actions	✅
Artefact de couverture téléchargeable	✅
La story Jira affiche le statut du build = Passed	✅

À la fin de l'**étape 3 — BUILD**, chaque branche de fonctionnalité doit automatiquement :

- Exécuter le lint + les tests + les vérifications de couverture
- Imposer des standards de qualité minimaux
- Remonter les résultats à la fois dans GitHub et dans Jira

Étape 4 — TEST (Vitest + Supertest)

Cette étape porte sur la vérification de la qualité et de la fiabilité de votre application à l'aide de **Vitest** (pour les tests unitaires) et de **Supertest** (pour les tests d'intégration).

Vous allez exécuter les tests existants, en comprendre la structure et, le cas échéant, en ajouter de nouveaux pour améliorer la couverture.

Objectif

- Vérifier que tous les endpoints et utilitaires fonctionnent correctement.
- Maintenir une **couverture minimale de 80 %** sur tous les fichiers source.

ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

- Valider automatiquement chaque Pull Request grâce au **workflow CI**.

1. Localiser les dossiers de tests dans votre projet

Ouvrez votre projet ZIP (Full_DevOps_Lab_Project_Final_Commented_EN.zip) et parcourez :

```
test/
├── version.test.js
├── info.test.js
├── boom.test.js
├── health.test.js
└── unit/
    ├── appInfo.test.js
    ├── status.test.js
    └── errorHandler.test.js
```

Rôle des dossiers :

Dossier	Rôle
test/	Tests d'intégration → simulent de vraies requêtes HTTP vers l'application Express
test/unit/	Tests unitaires → testent la logique isolée des fonctions de src/utils

2. Comprendre le framework de test

Le projet utilise :

- **Vitest** → framework de test compatible avec la syntaxe de Jest
- **Supertest** → permet d'envoyer de fausses requêtes HTTP à votre application Express

La configuration de Vitest est déjà incluse dans package.json :

```
"test": "vitest run",
"test:watch": "vitest",
"vitest": {
  "coverage": {
    "provider": "v8",
    "include": ["src/**/*.js"],
    "lines": 80,
    "functions": 80,
    "branches": 70,
    "statements": 80
  }
}
```

ESILV
ENGINEERING SCHOOL
DE VINCI PARIS

3. Lancer tous les tests en local

Pour exécuter tous les tests une fois :

```
npm test
```

Pour les exécuter en continu en mode watch (utile pendant le développement) :

```
npm run test:watch
```

Pour inclure l'analyse de couverture :

```
npm test -- --coverage
```

Sortie console attendue :

```
✔ All tests passed
Coverage summary:
Statements : 85%
Branches   : 72%
Functions  : 88%
Lines      : 84%
```

4. Tests d'intégration (endpoints de l'API)

Les tests d'intégration se trouvent directement dans le dossier test/.

Ils simulent de vrais appels HTTP vers votre application Express grâce à Supertest.

Exemple 1 — /version

Fichier : test/version.test.js

```
describe("GET /version", () => {
  it("returns package version as a non-empty string", async () => {
    const res = await request(app).get("/version");
    expect(res.status).toBe(200);
    expect(typeof res.body.version).toBe("string");
  });
});
```

Ce test vérifie la route /version dans src/routes/auto/version.route.js.

Exemple 2 — /info

Fichier : test/info.test.js

```
describe("GET /info", () => {
  it("returns app info", async () => {
    const res = await request(app).get("/info");
    expect(res.status).toBe(200);
    expect(res.body).toHaveProperty("name");
    expect(res.body).toHaveProperty("uptime");
  });
});
```

Ce test vérifie la route /info dans src/routes/auto/info.route.js, qui combine des utilitaires de src/utils/appInfo.js.

Exemple 3 — /boom

Fichier : test/boom.test.js

```
describe("GET /boom", () => {
  it("returns 500 with error payload", async () => {
    const res = await request(app).get("/boom");
    expect(res.status).toBe(500);
    expect(res.body.error).toBe(true);
  });
});
```

Cela garantit que la gestion des erreurs fonctionne correctement avec src/utils/errorHandler.js.

5. Tests unitaires (fonctions pures)

Les tests unitaires sont rangés dans test/unit/.

Ils importent et testent directement de petits utilitaires réutilisables.

Exemple 1 — formatStatus()

Fichier : test/unit/status.test.js

```
describe("formatStatus", () => {
  it("returns 'steady' when uptime ≥ 3600s", () => {
    expect(formatStatus(3600)).toBe("steady");
  });
});
```

Teste la logique de src/utils/status.js en garantissant une couverture de 100 % des branches.

Exemple 2 — getPackageInfo() et getRuntimeInfo()

Fichier : test/unit/appInfo.test.js

```
describe("getPackageInfo", () => {
  it("returns name and version", () => {
    const info = getPackageInfo();
    expect(info).toHaveProperty("name");
    expect(info).toHaveProperty("version");
  });
});
```

Valide la lecture des données de package.json dans src/utils/appInfo.js.

Exemple 3 — Gestionnaire d'erreurs

Fichier : test/unit/errorHandler.test.js

```
describe("errorHandler", () => {
  it("returns 500 with default message", () => {
    const res = mockRes();
    errorHandler({}, {}, res);
    expect(res.statusCode).toBe(500);
  });
});
```

Garantit une gestion correcte et une sortie d'erreur standardisée dans toute l'application.

6. Ajouter ou compléter des tests (pour une meilleure couverture)

Si vous modifiez ou ajoutez de la logique, vous devez aussi mettre à jour ou créer les fichiers de test correspondants.

Exemples :

- Nouvelle condition ajoutée dans src/utils/status.js → mettre à jour test/unit/status.test.js
- Nouvelle route /metrics → créer test/metrics.test.js en suivant la même structure que les autres.

Pour vérifier la couverture visuellement :

```
open coverage/index.html
```

Vous verrez un affichage par couleur (vert = testé, rouge = manquant).

7. Vérifier la couverture dans la CI/CD

À chaque fois que vous poussez votre branche ou ouvrez une Pull Request :

- GitHub Actions exécute automatiquement tous les tests.
- Si un test échoue ou si la couverture descend sous les seuils, le pipeline ❌ échouera et bloquera la fusion.

Dans GitHub → **onglet Actions → CI → Summary**, vérifiez :

- Tous les tests ont réussi
- L'artefact de couverture a été déposé
- Télécharger coverage-report.zip → ouvrir index.html

8. Dépannage

Problème	**Cause**	**Solution**
« app is not defined.»	Import manquant dans le fichier de test	Ajouter import app from "../src/app.js";
« Cannot read property version.»	Mauvais endpoint ou champ manquant	Revérifier la réponse de la route
Couverture inférieure à 80 %	Branche ou test négatif manquant	Ajouter des cas de tests unitaires
La CI échoue	Erreurs de lint ou de tests dans le code	Lancer npm run lint + npm test en local

9. Liste de vérification de fin d'étape

Élément à valider	**Résultat attendu**
Tous les tests d'intégration passent	✅
Tous les tests unitaires passent	✅
Couverture locale ≥ 80 %	✅
Étape de test du pipeline CI verte	✅
Artefact de couverture disponible dans GitHub Actions	✅
La story Jira met à jour automatiquement le statut du build	✅

Fin de l'étape 4 — TEST :

À ce stade, votre projet est entièrement vérifié : la fonctionnalité de chaque étudiant est validée à la fois par les tests locaux et par le pipeline CI.

Votre tableau Jira doit maintenant afficher toutes les stories dans « **Review** » ou « **Done** », avec les commits, les branches et les résultats de build liés.